TinyML: Bringing Machine Learning to Edge Devices

Abhishikth S Siby
Student
Department of Computer Applications
Mangalam College of Engineering
Ettumanoor, India
abhishikthsiby234@gmail.com

Anju John
Assistant Professor
Department of Computer Applications
Mangalam College of Engineering
Ettumanoor, India
anjujohn706@gmail.com

Archana Sunil
Student
Department of Computer Applications
Mangalam College of Engineering
Ettumanoor, India
archanasunil2182004@gmail.com

Abstract*—Tiny Machine Learning (TinyML) brings machine-learning inference onto severely constrained microcontrollers (MCUs) that typically operate within about one milliwatt of power and only kilobytes of RAM and flash. This paper surveys the TinyML architecture, development workflow, model families, and optimization methods—quantization, pruning, distillation, and compact-network design—together with the supporting hardware ecosystem (Arduino, ESP32, ARM Cortex-M, Raspberry Pi Pico). Beyond this review, the paper highlights a persistent gap in the literature: deployed TinyML models are typically static and cannot economically adapt once field conditions drift from training conditions. To address this, a conceptual framework, Adaptive Resource-Aware TinyML (ARA-TinyML), is proposed, pairing a lightweight on-device drift detector with a tiered, resource-bounded adaptation policy. An accelerometer/gyroscope-based accident-detection case study illustrates the pipeline. All comparisons with Cloud ML and Edge AI, and all optimization tables, are drawn from published literature or offered as theoretical projections; no new experiments were conducted.

***Keywords*—TinyML,Edge Computing, Embedded Machine Learning, Quantization, Pruning, Knowledge Distillation, Microcontrollers, Concept Drift, On-Device Learning, IoT.**

I. INTRODUCTION

Cloud-based machine-learning pipelines incur round-trip latency, consume bandwidth and energy, and expose private data as the number of connected sensors grows [13]. Edge computing addresses part of this problem by moving inference to a gateway or single-board computer closer to the data source. TinyML pushes the idea further still, running inference directly on the sensing microcontroller itself, frequently at well under one milliwatt of active power [1], [2].

Three developments made this practical. First, compression techniques—pruning, quantization, and knowledge distillation—now shrink trained networks by an order of magnitude with only modest accuracy loss [6]–[8]. Second, compact architectures built around depthwise-separable convolutions reduce the computational cost of on-device inference [9], [10], [40]. Third, mature embedded runtimes such as TensorFlow Lite Micro and CMSIS-NN allow models to run without an operating system or dynamic memory allocation [2], [21].

TinyML-enabled devices are expected to operate for months or years on small batteries, often with unreliable connectivity, while remaining both useful and privacy-preserving. Accordingly, this paper reviews TinyML architecture, model families, and optimization techniques; surveys representative hardware; presents an accident-detection case study; identifies a specific gap in post-deployment adaptability; and proposes a conceptual framework, ARA-TinyML, to address it. No new experiments were run: all numeric figures cited are attributed to the literature or explicitly labeled as theoretical projections.

II. BACKGROUND

Machine learning extracts statistical patterns from data; training is computationally expensive and usually centralized, whereas inference—TinyML's focus—is comparatively cheap, and supervised classification or regression on labeled sensor data dominates TinyML use cases [1], [5]. Edge computing distributes computation across a cloud–gateway–device continuum; TinyML represents the terminal case, where the "edge" is the MCU itself, lacking the operating system and megabyte-to-gigabyte memory available to gateway-class Edge AI [13], [18]. Embedded systems combine a processor core with small SRAM/flash memories and fixed peripherals under deterministic timing and minimal power, and TinyML models must generally fit within tens of kilobytes to a few megabytes of flash, often under 256–512 KB of SRAM [1], [18]. As the Internet of Things scales, streaming raw sensor data to the cloud becomes a bandwidth, energy, and latency bottleneck [13], [16]; TinyML instead transmits only compact results. TinyML combines these threads—running inference within roughly 1 mW and a few hundred kilobytes on an MCU [1], [14], [15]—and matured rapidly after 2019 through TensorFlow Lite Micro, the TinyML Foundation, and no-code tools such as Edge Impulse [2], [15], [30].

III. PROBLEM STATEMENT AND RESEARCH GAP

TinyML typically treats a deployed model as a fixed artifact: trained, compressed, flashed, and left running indefinitely without adjustment. This works for narrow, stationary tasks but breaks down once sensor-data distributions drift across users, seasons, or equipment wear, raising a central question: how can

a TinyML system remain accurate under field drift without exceeding MCU-class memory, latency, and energy budgets?

Four observations frame this gap. Static-by-default deployment remains the norm—train once, quantize, flash, with no built-in adjustment path [1], [2], [15], [19], [41].

"Reformable" or adaptive TinyML is still nascent: Rajapakse et al. note that on-device learning and adaptation are far less mature and standardized than static inference [23]. Training data rarely represents later field conditions, yet few systems detect this mismatch [16], [18], [20], [27]. Finally, the available options sit at two extremes—full local retraining, often infeasible on kilobyte-scale RAM, or a full cloud round trip, which reintroduces the costs TinyML is meant to avoid—leaving a cheap, bounded middle tier largely unexplored [18], [23], [26], [29].

The resulting gap: no widely adopted, resource-bounded method currently lets a deployed TinyML model detect drift and choose between no action, a cheap local adjustment, or an external update request, all within MCU budgets. Section XI proposes a framework aimed at this gap.

IV. OBJECTIVES

This paper aims to (1) describe TinyML's architecture and workflow; (2) survey model families and optimization techniques; (3) survey representative hardware platforms; (4) compare TinyML with Edge AI and Cloud ML on measurable dimensions; (5) ground the discussion in an accident-detection case study; (6) state a literature-supported research gap; and (7) propose a conceptual framework, clearly labeled as theoretical, that addresses this gap.

V. ARCHITECTURE AND WORKFLOW

A typical TinyML pipeline runs from sensing to action: sensors acquire raw signals, which are filtered, normalized, windowed, and reduced to features (e.g., MFCCs or simple statistics) before being passed to a quantized model that emits a label or set of probabilities; predictions are then debounced into an application-level decision such as an LED, BLE alert, or relay action, with only compact results ever leaving the device [1], [2], [3], [5], [13]. The rate at which sensors sample trades signal fidelity for energy consumption, and preprocessing itself is compute-bounded on MCU-class hardware.

The workflow that produces this pipeline begins offline: problem definition and data collection, followed by model design and training on a GPU or in the cloud (typically with TensorFlow/Keras or PyTorch), optimization through quantization, pruning, and distillation, and conversion to a device-ready format such as .tflite [2]. The model is then flashed to the target MCU, where it performs on-device inference alongside lightweight monitoring; a further update step—where supported—remains the open gap discussed in Section III. No-code tools such as Edge Impulse now let a working image classifier be built end-to-end in roughly an hour [30].

VI. MODELS AND OPTIMIZATION TECHNIQUES

Small dense networks are adequate for low-dimensional feature inputs [1]. Convolutional networks built from depthwise-separable convolutions—MobileNet/V2 and XiNet among them—handle spatial or temporal data at a much lower multiply-accumulate cost than standard convolutions [9], [10], [40]; the DS-CNN architecture, a widely cited keyword-spotting example, outperforms plain CNN, RNN, and dense baselines on-device [5]. Classical models such as SVMs, decision trees, and k-NN remain competitive for low-dimensional tasks and offer deterministic memory use, as in an SVM built on 56 temporal accelerometer features for fall detection [24]. Transfer learning further lowers on-device training cost by reusing pretrained representations [31], [33], while neurosymbolic and probabilistic-inference approaches remain an emerging, still underexplored direction for MCU-class hardware [39].

Four optimization families make these models fit MCU budgets. Quantization reduces numeric precision, commonly from FP32 to INT8; integer-only inference removes the need for a fast floating-point unit and, combined with quantization-aware training, preserves near-baseline accuracy [8], with mixed-precision accelerators such as the RISC-V STAR-MAC unit further improving integer-inference efficiency [37]. Pruning removes low-importance weights or channels; Deep Compression combined pruning, quantization, and Huffman coding to achieve 35–49× storage reduction on the networks it evaluated [6], and structured pruning is generally preferred on MCUs that lack sparse-kernel support. Distillation trains a small student network on a larger teacher's softened outputs, improving the accuracy attainable at small model sizes [7]. Hardware-co-designed architecture search extends this further: MCUNet's TinyNAS and TinyEngine enable ImageNet-scale classification under roughly 1 MB of SRAM [3], and patch-based inference in MCUNetV2 cuts peak activation memory still further [4].

Table I summarizes these techniques.

TABLE I. OPTIMIZATION TECHNIQUES — LITERATURE-BASED SUMMARY

Technique	Target	Typical Effect	Ref.
INT8 quantization	Flash/RAM/CPU	~4× smaller than FP32; near-baseline w/ QAT	[8]
Pruning + Huffman coding	Flash/ storage	35–49× reduction, no measured loss	[6]
Knowledge distillation	Student accuracy	Improves small-model accuracy	[7]
Depthwise-sep. convs / XiNet	MACs, latency	Large MAC reduction vs. std. convs	[9],[40]
TinyNAS + TinyEngine	Peak SRAM	ImageNet-scale under ~1MB SRAM	[3]
Patch-based inference	Peak activation	Further SRAM reduction	[4]

VII. HARDWARE PLATFORMS

MCUs integrate a processor core, SRAM, and flash on a single chip without an MMU or a full operating system, and this

combination defines the TinyML hardware class [1], [18]. The Arduino Nano 33 BLE Sense (Cortex-M4) is a common entry point for education and prototyping [1], [2]; the ESP32 adds Wi-Fi/BLE connectivity at the cost of higher idle power; ARM Cortex-M cores (M0–M55) dominate deployments, aided by the CMSIS-NN kernel library [2], [21]; and the Raspberry Pi Pico (RP2040, dual Cortex-M0+) offers a low-cost option compatible with INT8 TensorFlow Lite Micro models. Table II lists typical specifications.

TABLE II. ILLUSTRATIVE HARDWARE PLATFORMS (TYPICAL SPECS)

Platform	Core	SRAM	Flash	Use
Arduino Nano 33 BLE	Cortex-M4F	~256KB	~1MB	KWS/IMU [1],[2]
ESP32	Xtensa LX6 x2	~520KB	4MB+	Connected sensing
Cortex-M4/M33	ARM M4/M33	128K–1M	256K–2M	CMSIS-NN [21]
Pi Pico	Cortex-M0+ x2	264KB	2MB	Low-cost proto.

VIII. APPLICATIONS

Reported TinyML applications span several domains. In healthcare, on-body signal analysis and assistive navigation avoid streaming biometric data off-device [13], [33], [36]. Always-on keyword spotting, commonly via DS-CNN on the Speech Commands dataset, supports voice interfaces [5], [22]. Wrist- and body-worn IMU classification, including time-of-flight-based doorway activity recognition, supports activity monitoring [24], [25], [42]. Environmental monitoring—air quality, structural vibration, microplastic detection, and solar-yield forecasting—benefits from local anomaly flags that extend battery life by avoiding continuous telemetry [13], [16], [34], [43]. Industrial predictive maintenance from vibration and acoustic signals has been reviewed holistically across the TinyML stack [16], [18], [27]. Smart-agriculture and smart-home use cases include local soil and environment classification and privacy-preserving presence, gesture, and voice control [13], [15], [16]. Accident detection, the focus of the case study below, requires safety-critical, bounded-latency alerting independent of connectivity [24], [25].

IX. CASE STUDY: ACCIDENT DETECTION

Falls and accidents benefit from rapid, connectivity-independent local alerting rather than a cloud round trip. A representative pipeline collects accelerometer and gyroscope data, filters and preprocesses it, extracts features, and classifies the result as normal or a possible accident before triggering an alert. Windowed tri-axial data is either hand-featurized for an SVM or fed directly to a small CNN or LSTM, and a confirmation step—such as detecting stillness following an impact—reduces false positives [24], [25]. Reported figures, not re-verified here, illustrate the range of achievable performance: an SVM using 56 temporal IMU features reported high-90s percent accuracy on a public dataset [24], while a CNN-LSTM model reported under 130 KB of flash (about 97 KB on an STM32F407), under 60 ms inference latency (about 36 ms measured), over 99% accuracy, and near-99% F1, with modest accuracy loss when transferred across datasets [25]. Table III maps the sensing-to-action pipeline.

TABLE III. SENSOR → INPUT → TASK → PREDICTION → ACTION

Sensor	Input	ML Task	Predict.	Action
Accel.	Windowed x,y,z	Feature extract./raw-CNN	Feature vec.	→classifier
Gyro.	Windowed ω	Orient.-change feat.	Rot. feat.	Fused w/ accel.
Fused IMU	Combined vec.	Binary classification	Label+co nf.	Confirm logic
—	Confirmed event	Alerting (rule-based)	"Accident "	BLE/buzze r

X. COMPARISON: CLOUD ML, EDGE AI, AND TINYML

Table IV summarizes, at a qualitative and literature-informed level, how TinyML differs from Cloud ML and gateway-class Edge AI across location, latency, power, connectivity, privacy, hardware, bandwidth, and typical applications [1], [11], [13], [14], [18], [21], [35]. TinyML's advantages are clearest precisely where latency, privacy, bandwidth, and energy constraints are tightest, which is also where safety-relevant tasks such as accident detection tend to sit. This comparison is a conceptual synthesis of survey literature rather than a controlled benchmark run for this paper.

TABLE IV. CLOUD ML VS. EDGE AI VS. TINYML (QUALITATIVE)

Dimension	Cloud ML	Edge AI	TinyML
Location	Central server	Gateway/SBC	On the MCU
Latency	RTT-dependent	Low, local net.	Very low [11],[21]
Power	High (server)	Moderate (W)	Sub-mW–mW [1],[14]
Connectivit y	Always req.	Usually req.	Not needed
Privacy	Data leaves site	Often on-prem	Stays on device [13],[35]
Hardware	GPU/TPU, GB RAM	Multi-core, MB–GB	MCU, KB–MB [1],[18]
Apps	LLMs, batch analytics	Video analytics	KWS, activity, accident [5],[24]

XI. PROPOSED FRAMEWORK: ARA-TINYML

To address the gap identified in Section III without assuming either permanent staticness or unconstrained retraining and connectivity, this paper proposes Adaptive Resource-Aware TinyML (ARA-TinyML), a conceptual proposal in the spirit of the still-underdeveloped "reformable TinyML" direction [23]; it has not been implemented or tested here.

ARA-TinyML has four components. The existing static, quantized model continues to perform primary inference. A lightweight drift detector tracks a small number of running statistics—for example, an exponentially weighted moving average of prediction confidence—rather than storing raw history, so that sustained shifts, not noisy individual readings, trigger a drift signal [23]. A resource-aware scheduler then checks available energy, RAM, and latency headroom and chooses among three tiers: no action; a cheap partial on-device adaptation, such as recalibrating the last layer or bias terms from

a small locally buffered sample set, echoing transfer-learning-style reuse [31]; or flagging the device for an external update the next time connectivity is available, in line with distributed and federated on-device learning directions [26], [29]. Finally, a versioned model store with rollback reverts any adaptation that regresses post-update performance, mitigating the update-integrity risks discussed in Section XIII.

Table V compares the static baseline with the expected behavior of ARA-TinyML; all entries are design-rationale estimates offered as hypotheses for future empirical validation, not measured results.

TABLE V. STATIC TINYML VS. ARA-TINYML (THEORETICAL/EXPECTED)

Parameter	Static Baseline	ARA-TinyML (Expected)
Accuracy (stable)	Baseline [5],[8],[9]	Equivalent (Tier 1 = no change)
Accuracy (drift)	Degrades [16],[18],[23]	Partial recovery (hypothesis)
Latency	Baseline [11],[21],[25]	Unchanged; small Tier-2 cost
RAM	Baseline activation	+tens of bytes (drift stats)
Flash	Baseline size	+detector code + rollback copy
Energy	Baseline/cycle	Unchanged T1; small T2 [28]
Connectivity	None needed	Only Tier 3, opportunistic

XII. ADVANTAGES AND LIMITATIONS

TinyML's principal advantages include low latency without a network round trip [11], [21], [25]; sub-milliwatt-to-milliwatt active power enabling multi-month or multi-year battery life [1], [14]; privacy, since raw data need not leave the device [13], [35]; lower bandwidth and infrastructure cost through compact alerts [16]; offline operability suited to safety-critical use [24], [25]; and low hardware cost at scale [15].

These benefits come with real constraints. Severe resource limits force constant trade-offs between accuracy and footprint [1], [18], and compression-accuracy trade-offs require careful, task-specific tuning [6], [8]. Tooling and hardware remain fragmented across MCU vendors [15], [18], [41], and data scarcity for niche field conditions increases drift risk [16], [23]. No standardized approach to on-device adaptation yet exists—the gap Section XI targets [23]—and benchmarking itself remains fragmented beyond MLPerf Tiny's limited set of reference tasks [11], [17].

XIII. SECURITY, PRIVACY, AND FUTURE SCOPE

Local inference improves privacy relative to cloud pipelines [13], and privacy-preserving TinyML techniques for sensitive domains such as consumer healthcare are an active research area [35]. New risks specific to this setting include physical extraction of firmware or model weights, update-integrity risks for any post-deployment adaptation mechanism (mitigated in ARA-TinyML through versioned rollback), and constrained cryptographic capability on MCUs that limits standard update-authentication schemes [1], [15], [18].

Building on this, promising directions for future work include empirical validation of resource-bounded adaptation such as ARA-TinyML on real hardware and real drift [23]; standardized adaptation benchmarks extending MLPerf Tiny [11]; deeper hardware–software co-design continuing MCUNet-style efforts [3], [4], [37], [38]; federated and distributed learning across devices at the extreme edge [26], [29]; lightweight update-authentication primitives suited to MCUs [17]; and continued maturation of neurosymbolic and probabilistic TinyML approaches [39].

XIV. EVALUATION METRICS AND BENCHMARKING CONSIDERATIONS

Assessing a TinyML system requires more than reporting classification accuracy. Because deployment targets are bounded by flash, SRAM, latency, and energy budgets, a meaningful evaluation reports accuracy jointly with flash footprint, peak SRAM, inference latency, and energy per inference [1], [14], [18]. Any one of these metrics reported in isolation can hide a violated constraint: a model that is accurate on a workstation may still exceed the RAM of the intended MCU, and a model that fits in memory may still miss a real-time deadline. The literature accordingly frames TinyML design as a multi-objective trade-off rather than a single-metric optimization [1], [18].

Benchmarking practice is still maturing. MLPerf Tiny provides a common set of reference tasks for static inference [11], but the literature describes benchmarking as fragmented beyond that limited set of tasks [11], [17]. Energy measurement adds a further difficulty, and energy profiling techniques designed specifically for extreme-edge devices are themselves an active research topic [28]. For this reason, the comparisons in this paper are kept qualitative, and every numeric value is attributed to its original source.

For safety-relevant applications such as the accident-detection case study, aggregate accuracy is also insufficient. Missed detections and false alarms carry different costs: a missed event defeats the purpose of the system, whereas repeated false alarms can erode user trust and waste battery through unnecessary alerts. The confirmation step described earlier, such as checking for stillness after an impact, exists to reduce false positives [24], [25], and the cross-dataset behavior reported in [25] is a better indicator of expected field performance than in-dataset accuracy alone.

A future empirical study of ARA-TinyML would need metrics specific to adaptation, none of which are measured in this paper. Candidate measures include drift-detection delay, the false-alarm rate of the drift detector, accuracy recovered after a Tier-2 adjustment relative to the static baseline, the additional RAM and flash consumed by the detector and the rollback copy, energy per adaptation cycle, and the frequency with which rollback is triggered. Because standardized adaptation benchmarks do not yet exist [23], such a study would also have to specify its drift scenarios, for example changes across users, seasonal variation, and equipment wear, so that results can be reproduced and compared.

XV. DEPLOYMENT AND PRACTICAL CONSIDERATIONS

Moving a model from a training environment to a microcontroller involves choices that are largely invisible at the algorithm level. Toolchains and hardware remain fragmented across vendors [15], [18], [41], so a pipeline validated on one MCU family may need rework for another. Runtimes such as TensorFlow Lite Micro and kernel libraries such as CMSIS-NN reduce this burden for supported targets [2], [21], and no-code environments such as Edge Impulse shorten the path from data collection to a deployed classifier [30].

Memory planning deserves early attention. Peak activation memory can be as limiting as model size, which is why hardware-aware architecture search and patch-based inference target SRAM directly [3], [4]. Quantization choices must also suit the target runtime, since integer-only inference removes the dependence on a floating-point unit [8]. Architectures designed for commodity MCUs [12] and approximate accelerator designs for resource-constrained devices [38] offer further routes to meeting these limits.

Power management is a system-level concern. The sensor sampling rate trades signal fidelity for energy consumption, and on-device preprocessing is itself compute-bounded, so an end-to-end energy budget should cover sensing, feature extraction, inference, and any radio transmission rather than inference alone [1], [14]. Transmitting only compact results instead of raw data reduces bandwidth and radio use [13], [16], which is one reason on-device inference supports multi-month or multi-year battery life [1], [14].

Field maintenance introduces its own requirements. Any adaptation or update mechanism, including the Tier-3 external update request proposed in ARA-TinyML, depends on intermittent connectivity and on update-authentication schemes that fit within the constrained cryptographic capability of MCUs [1], [15], [18]. Versioned model storage with rollback lets a device recover from an adaptation that degrades performance, but it consumes additional flash that must be budgeted alongside the base model. In privacy-sensitive settings such as consumer healthcare, keeping raw data on the device remains a principal benefit [13], [35], although it does not remove the need to protect firmware and model weights from physical extraction.

Finally, evaluation before deployment should be performed on the intended MCU, or an equivalent board, rather than on desktop emulation alone, because latency, memory behavior, and energy differ between the two environments. The interaction between weight quantization and on-device or federated learning has been examined in an IoT case study [32], and it is directly relevant to any Tier-2 adaptation applied to a quantized model.

XVI. DISCUSSION AND CONCLUSION

The engineering foundations of TinyML—compression, compact architectures, embedded runtimes, and static-inference benchmarking—are comparatively mature and well verified [2], [3], [6], [8], [9], [11]. Its advantages over Cloud ML and Edge AI are strongest precisely where latency, privacy, bandwidth, and energy constraints are tightest, which is also where safety-relevant tasks such as accident detection sit. Post-deployment adaptation, by contrast, remains comparatively underdeveloped. ARA-TinyML is offered here as one resource-respecting way to partially close that gap, with Table V read as testable hypotheses rather than findings, pending a controlled implementation and field study.

In summary, this paper has reviewed TinyML's architecture, workflow, model families, and optimization techniques; compared TinyML with Cloud ML and Edge AI on measurable dimensions; and grounded the discussion in a literature-based accident-detection case study. It identified a specific research gap—the near-absence of standardized, resource-bounded post-deployment adaptation—and proposed ARA-TinyML, a conceptual, tiered drift-detection-and-adaptation framework explicitly labeled as theoretical. All prior-work figures are literature-attributed; future work should empirically validate resource-bounded adaptation on real MCU hardware.

REFERENCES

[1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol, CA: O'Reilly Media, 2019.

[2] R. David et al., "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems," Proc. MLSys, vol. 3, 2021, pp. 800–811.

[3] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny Deep Learning on IoT Devices," NeurIPS, vol. 33, 2020.

[4] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "MCUNetV2: Memory-Efficient Patch-Based Inference for Tiny Deep Learning," NeurIPS, 2021.

[5] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," arXiv:1711.07128, 2017.

[6] S. Han, H. Mao, and W. J. Dally, "Deep Compression," Int. Conf. Learning Representations (ICLR), 2016.

[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.

[8] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," Proc. CVPR, 2018, pp. 2704–2713.

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2," Proc. CVPR, 2018, pp. 4510–4520.

[10] A. G. Howard et al., "MobileNets," arXiv:1704.04861, 2017.

[11] C. Banbury et al., "MLPerf Tiny Benchmark," NeurIPS Datasets and Benchmarks Track, 2021.

[12] C. Banbury et al., "MicroNets: NN Architectures for TinyML on Commodity MCUs," Proc. MLSys, 2021.

[13] R. Sanchez-Iborra and A. F. Skarmeta, "TinyML-Enabled Frugal Smart Objects," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.

[14] P. P. Ray, "A Review on TinyML: State-of-the-Art and Prospects," J. King Saud Univ. – CIS, vol. 34, no. 4, pp. 1595–1623, 2022.

[15] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, "A Comprehensive Survey on TinyML," IEEE Access, vol. 11, pp. 96892–96922, 2023, doi: 10.1109/ACCESS.2023.3294111.

[16] L. Dutta and S. Bharali, "TinyML Meets IoT: A Comprehensive Survey," Internet of Things, vol. 16, 2021.

[17] L. Capogrosso, F. Cunico, D. S. Cheng, F. Fummi, and M. Cristani, "A Machine Learning-Oriented Survey on Tiny Machine Learning," IEEE

Access, vol. 12, pp. 23406–23426, 2024, doi: 10.1109/ACCESS.2024.3365349.

[18] S. S. Saha, S. S. Sandha, and M. Srivastava, “Machine Learning for Microcontroller-Class Hardware: A Review,” IEEE Sensors J., vol. 22, no. 22, pp. 21362–21390, 2022.

[19] V. Tsoukas, A. Gkogkidis, E. Boumpa, and A. Kakarountas, “A Review on the Emerging Technology of TinyML,” ACM Comput. Surv., vol. 56, no. 10, 2024.

[20] M. T. Lê, P. Wolinski, and J. Arbel, “Efficient Neural Networks for Tiny Machine Learning: A Comprehensive Review,” arXiv:2311.11883, 2023.

[21] L. Lai, N. Suda, and V. Chandra, “CMSIS-NN: Efficient NN Kernels for Arm Cortex-M CPUs,” arXiv:1801.06601, 2018.

[22] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” arXiv:1804.03209, 2018.

[23] V. Rajapakse, I. Karunanayake, and N. Ahmed, “Intelligence at the Extreme Edge: A Survey on Reformable TinyML,” ACM Comput. Surv., vol. 55, pp. 1–30, 2023.

[24] O. Durgun et al., “Fall Detection Systems Supported by TinyML and Accelerometer Sensors,” Int. Scientific and Vocational Studies J., vol. 7, no. 1, pp. 55–61, 2023.

[25] L.-C. Duan, N.-N. Minh, T.-C. Dung, and T.-T. Linh, “Energy-Efficient TinyML Approach for Wearable Fall Detection on Edge Devices,” Int. J. Technology, vol. 17, no. 3, pp. 919–935, 2026.

[26] W. Villegas-Ch, R. Gutierrez, A. Maldonado Navarro, and A. Mera-Navarrete, “Optimizing Federated Learning on TinyML Devices for Privacy Protection and Energy Efficiency in IoT Networks,” IEEE Access, vol. 12, pp. 174354–174370, 2024, doi: 10.1109/ACCESS.2024.3503516.

[27] E. J. Njor, M. A. Hasanpour, J. Madsen, and X. Fafoutis, “A Holistic Review of the TinyML Stack for Predictive Maintenance,” IEEE Access, vol. 12, pp. 184861–184882, 2024, doi: 10.1109/ACCESS.2024.3512860.

[28] K. Müller, J. Weidner, N. Franchi, and P. Wägemann, “TinyEP: TinyML-Enhanced Energy Profiling for Extreme Edge Devices,” IEEE Access, vol. 12, pp. 193747–193762, 2024, doi: 10.1109/ACCESS.2024.3520089.

[29] M. Gulati, K. Zandberg, Z. Huang, G. Wunder, C. Adjih, and E. Baccelli, “TDMiL: Tiny Distributed Machine Learning for Microcontroller-Based Interconnected Devices,” IEEE Access, vol. 12, pp. 167810–167826, 2024, doi: 10.1109/ACCESS.2024.3492921.

[30] R. Berta, A. Dabbous, L. Lazzaroni, and D. P. Pau, “Developing a TinyML Image Classifier in an Hour,” IEEE Open J. Industrial Electronics Society, vol. 5, pp. 946–960, 2024, doi: 10.1109/OJIES.2024.3451959.

[31] A. Hayajneh, M. Hafeez, S. A. R. Zaidi, and D. McLernon, “TinyML Empowered Transfer Learning on the Edge,” IEEE Open J. Communications Society, vol. 5, pp. 1656–1672, 2024, doi: 10.1109/OJCOMS.2024.3373177.

[32] N. L. Giménez, J. Lee, F. Freitag, and H. Vandierendonck, “The Effects of Weight Quantization on Online Federated Learning for the IoT: A Case Study,” IEEE Access, vol. 12, pp. 5490–5502, 2024, doi: 10.1109/ACCESS.2024.3349557.

[33] S. Ponnada, T. K. Tak, P. R. Kshirsagar, S. Rao, et al., “Expanding Applications of TinyML in Versatile Assistive Devices,” IEEE Access, vol. 12, pp. 168328–168338, 2024, doi: 10.1109/ACCESS.2024.3496791.

[34] A. Cerioli, L. Petrosino, D. Sasso, C. Laroche, T. Piechowiak, L. Pezzarossa, M. Merone, L. Vollero, and A. Sabatini, “Efficient Detection of Microplastics on Edge Devices with Tailored Compiler for TinyML Applications,” IEEE Access, vol. 13, pp. 90970–90982, 2025, doi: 10.1109/ACCESS.2025.3567816.

[35] S. Aanjankumar, M. K. Muchahari, S. Urooj, I. Kaur, R. K. Dhanaraj, H. A. Mengash, S. Poonkuntran, and P. R. Kaveri, “Enhanced Consumer Healthcare Data Protection Through AI-Driven TinyML and Privacy-Preserving Techniques,” IEEE Access, vol. 13, pp. 97428–97440, 2025, doi: 10.1109/ACCESS.2025.3573076.

[36] C. Yahyati, I. Lamaakal, Y. Maleh, K. El Makkaoui, I. Ouahbi, M. Almousa, and A. A. Abd El-Latif, “A Systematic Review of State-of-the-Art TinyML Applications in Healthcare, Education, and Transportation,” IEEE Access, vol. 13, pp. 204513–204562, 2025, doi: 10.1109/ACCESS.2025.3633575.

[37] E. Manca, L. Urbinati, and M. R. Casu, “Accelerating Mixed-Precision QNN Inference on RISC-V MCUs With the STAR-MAC Unit,” IEEE Access, vol. 13, pp. 208533–208548, 2025, doi: 10.1109/ACCESS.2025.3641382.

[38] P. Thejaswini, G. Suresh, V. Chiraag, and S. Nandi, “Approximate CNN Hardware Accelerators for Resource Constrained Devices,” IEEE Access, vol. 13, pp. 12542–12553, 2025, doi: 10.1109/ACCESS.2025.3529668.

[39] J. Leslin, M. Trapp, and M. Andraud, “Hardware-Efficient Tractable Probabilistic Inference for TinyML Neurosymbolic AI Applications,” Proc. IEEE Int. Conf. Omni-Layer Intelligent Systems (COINS), 2025, doi: 10.1109/COINS65080.2025.11125733.

[40] A. Ancilotto, F. Paissan, and E. Farella, “XiNet: Efficient Neural Networks for tinyML,” Proc. IEEE/CVF ICCV, 2023, pp. 16968–16977.

[41] A. Elhanashi, P. Dini, S. Saponara, and Q. Zheng, “Advancements in TinyML: Applications, Limitations, and Impact on IoT Devices,” Electronics, vol. 13, no. 17, art. 3562, 2024, doi: 10.3390/electronics13173562.

[42] S. Taheri Moghadar, R. Gandolfi, and E. Torti, “TinyML-Based Real-Time Doorway Activity Recognition with a Time-of-Flight Sensor,” Electronics, vol. 14, no. 17, art. 3533, 2025.

[43] “Intelligent Solar Forecasts: Modern Machine Learning Models and TinyML Role for Improved Solar Energy Yield Predictions,” IEEE Access, 2024, doi: 10.1109/ACCESS.2024.3354703.